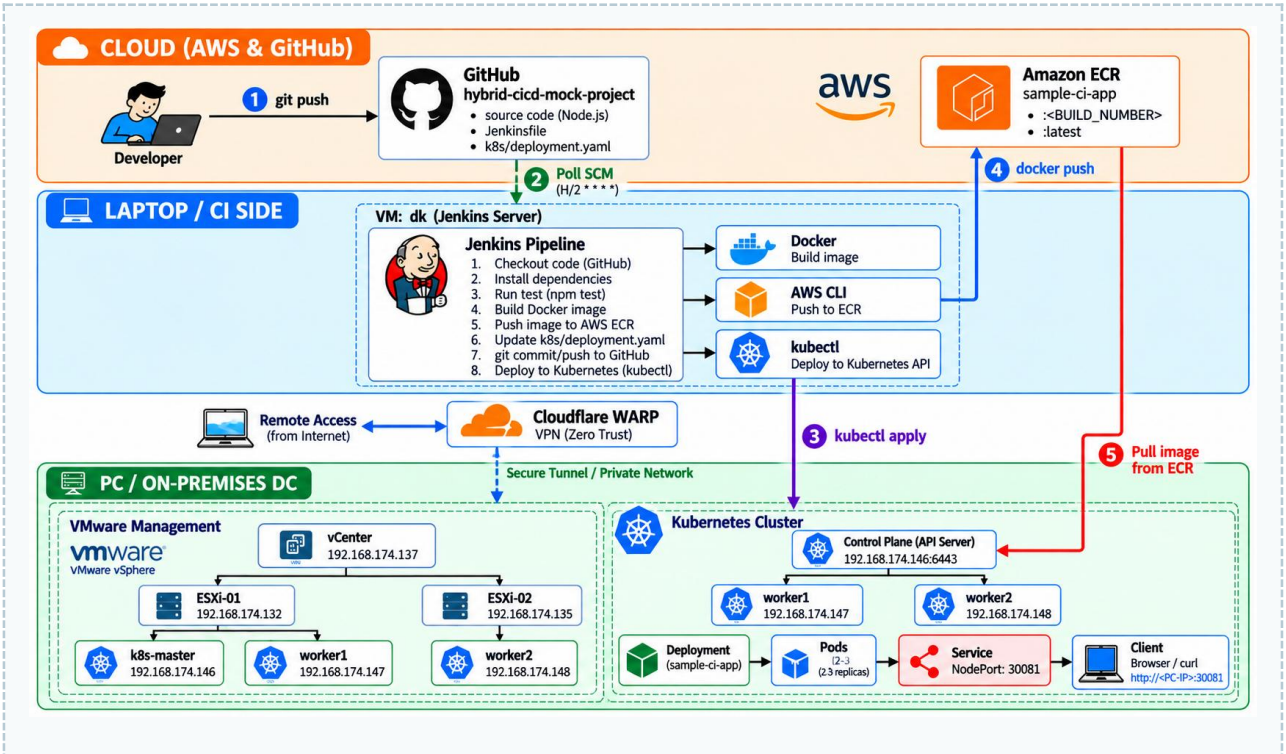
BÁO CÁO THỰC HÀNH
HYBRID CI/CD MOCK PROJECT

VMWARE ESXi • JENKINS • DOCKER • GITHUB • AMAZON ECR • KUBERNETES

Developer → GitHub (main) → Jenkins Poll SCM → npm test
→ Docker Build → Amazon ECR (:BUILD_NUMBER)
→ Jenkins GitOps Commit [skip ci] → kubectl apply
→ Kubernetes trên VMware → NodePort 30081

PHẠM VI	THÔNG TIN THỰC TẾ
Môi trường CI	Jenkins Docker trên VM Ubuntu dk — 192.168.232.170
Môi trường CD	Kubernetes 3 Nodes trên VMware ESXi (192.168.174.146–148)
Private Registry	Amazon ECR — ap-southeast-2 / sample-ci-app
Bản triển khai được nghiệm thu	Jenkins Build #52 → Docker image :52 → Kubernetes rollout



Hình -01 — Sơ đồ kiến trúc Hybrid CI/CD tổng thể

MỤC LỤC

1. TỔNG QUAN KIẾN TRÚC VÀ THÔNG SỐ HỆ THỐNG 4
1.1. Mô hình triển khai Hybrid 4
1.2. Bảng thông số các thành phần 4
1.3. Tiêu chí hoàn thành 5
2. CHUẨN BỊ VMWARE ESXI VÀ NETWORK 5
2.1. Kiểm tra hạ tầng vSphere 5
2.2. Dung lượng Datastore ESXi-01 5
2.3. Máy ảo Ubuntu cho Kubernetes 6
2.4. Kết nối mạng đến Kubernetes API 6
3. CÀI ĐẶT MÁY CI-LAB: DOCKER VÀ JENKINS 7
3.1. Chuẩn bị CI Server dk 7
3.2. Kiểm tra Docker Engine và Jenkins Container 7
3.3. Jenkins Web UI và Job 7
3.4. Các công cụ CI trong Jenkins 8
4. CHUẨN BỊ MÃ NGUỒN GITHUB VÀ AWS ECR 8
4.1. Repository và cấu trúc thư mục 8
4.2. Ứng dụng Node.js mẫu 9
4.3. Dockerfile sử dụng non-root 9
4.4. Amazon ECR Private Registry 10
4.5. Kiểm tra quyền truy cập ECR không hiển thị secret 10
5. CẤU HÌNH JENKINS PIPELINE VÀ GITHUB POLL SCM 10
5.1. Tạo Pipeline Job dùng Jenkinsfile từ GitHub 10
5.2. Trigger Poll SCM 11
5.3. Jenkins Credentials 12
5.4. Các stage chính trong Jenkinsfile 12
5.5. Ngăn vòng lặp CI bởi GitOps commit 12
6. TRIỂN KHAI KUBERNETES CLUSTER TRÊN VMWARE 14
6.1. Cấu trúc Cluster 14
6.2. Xác minh Nodes / System Pods 14
6.3. Mạng Kubernetes và DNS nội bộ 15
6.4. Tài nguyên và trạng thái Node 15
7. KUBERNETES MANIFESTS VÀ JENKINS DEPLOYMENT 16
7.1. Namespace và tài nguyên ứng dụng 16
7.2. Deployment YAML — các phần quan trọng 16
7.3. Service NodePort 30081 16
7.4. ECR Pull Secret và CronJob làm mới token 17
7.5. Deployment Strategy và Rollout 17
8. KIỂM THỬ TOÀN BỘ LUỒNG HYBRID CI/CD 18
8.1. Kịch bản End-to-End 18

8.2. Quan sát Jenkins Build #52 18
8.3. Quan sát Amazon ECR 19
8.4. Quan sát GitOps commit và khả năng chống lặp 19
8.5. Quan sát Kubernetes rollout và HTTP 20
8.6. Bảng nghiệm thu End-to-End 21
9. VẬN HÀNH, BẢO MẬT VÀ XỬ LÝ LỖI THƯỜNG GẶP 22
9.1. Pod Self-healing 22
9.2. Planned Worker Failover 22
9.3. Hardening Container / Jenkins Credentials 22
9.4. Các lưu ý tài nguyên và sự cố 23
9.5. Kiểm thử phục hồi sau thời gian shutdown dài hạn (ECR Token Expiration & Recovery) 24
9.6. Tối ưu Jenkins CI/CD Pipeline – Tránh Lập Build 24
10. PHỤ LỤC: FILE CẤU HÌNH VÀ BẢNG LỆNH TRA CỨU 25
10.1. Đường dẫn file và phiên bản tham chiếu 25
10.2. File k8s/deployment.yaml — bản cấu hình tại commit 025fab4 25
10.3. File Dockerfile, service.yaml và package.json 27
10.4. Jenkinsfile — các đoạn cấu hình trọng tâm 27
10.5. Bảng lệnh tra cứu nhanh 28
11. Kết nối mạng Hybrid thông qua Cloudflare Zero Trust/WARP 29
11.1 Các thành phần triển khai: 29

1. TỔNG QUAN KIẾN TRÚC VÀ THÔNG SỐ HỆ THỐNG

Mục tiêu. Tự động hóa chuỗi CI/CD: Developer push code lên GitHub → Jenkins tự động build/test → đóng gói Docker → push vào Amazon ECR → cập nhật GitOps manifest → Jenkins triển khai Kubernetes trên VMware → ứng dụng truy cập qua NodePort 30081.

1.1. Mô hình triển khai Hybrid

CI được vận hành trên máy Ubuntu dk (Laptop). Cluster Kubernetes chạy trên các VM thuộc VMware ESXi (PC). AWS được sử dụng làm private container registry; Cloudflare Zero Trust/WARP phục vụ truy cập từ xa vào Home Lab, không phải điều kiện bắt buộc trên tuyến Jenkins → Kubernetes.

```
GitHub / main
└── Jenkins sample-ci-pipeline (dk: 192.168.232.170)
    ├── Install dependencies / npm test
    ├── Docker build :52 ──────────→ Amazon ECR / sample-ci-app
    ├── Commit GitOps manifest [skip ci]
    └── kubectl apply ──────────────→ Kubernetes API: .146:6443
                        ├── k8smaster .146
                        ├── worker1   .147
                        └── worker2   .148 → NodePort 30081
```

GitHub (Cloud)

Thông số	Giá trị
Repository	hybrid-cicd-mock-project
Branch	main
Nội dung chính	source code (Node.js) Jenkinsfile k8s/deployment.yaml
Jenkins / Poll SCM	H/2 * * * * (kiểm tra thay đổi mỗi ~2 phút)

AWS (Cloud)

Thông số	Giá trị
Region	ap-southeast-2
Dịch vụ sử dụng	Amazon ECR, IAM
ECR Repository	sample-ci-app
Image tag	<BUILD_NUMBER> :latest
Xác thực ECR	ECR Pull Secret (tự động refresh)

Jenkins (Laptop – CI Side)

Thông số	Giá trị
Hostname	dk
IP Address	192.168.232.170
Container	jenkins/jenkins:lts (custom) (docker-cli, aws-cli, kubectl, node, git)
Chức năng	Checkout → Test → Build Docker → Push ECR → Update k8s manifest → Deploy

Kubernetes (trên VMware ESXi)

Thông số	Giá trị
Control Plane	mk8s — 192.168.174.146
Worker 1,2	1 — 192.168.174.147, 2 — 192.168.174.148
Ansible	Playbook.ini, control 2 worker
Deployment	sample-ci-app (2–3 replicas)
Service	sample-ci-app (NodePort: 30081)

VMware vSphere (PC)

Thông số	Giá trị
ESXi-01	192.168.174.132
ESXi-02	192.168.174.135
ESXi-01 datastore	worker2
ESXi-02 datastore	worker1
Datastore NFS	192.168.174.138, k8s-master

Cloudflare (Remote Access)

Thông số	Giá trị
Dịch vụ	Cloudflare Zero Trust / WARP
Mục đích	Truy cập từ xa vào Home Lab (vCenter/ESXi/K8s)
Kết nối	WARP client (Laptop) → cloudflared (PC tại nhà)
Mạng	Kết nối tới 192.168.174.0/24 → 192.168.232.0/24
Lưu ý	Sử dụng tunnel/private network, không mở port

Hình 01-01 — Sơ đồ CI/CD Hybrid và mạng kết nối

1.2. Bảng thông số các thành phần

Thành phần	Địa chỉ / thông số	Vai trò
vCenter	192.168.174.137	Quản trị vSphere (không cần bật để thu thập bằng chứng còn lại)
ESXi-01 / ESXi-02	192.168.174.132 / 192.168.174.135	Hypervisor chạy lab
CI VM — dk	192.168.232.170 • Ubuntu • Jenkins :8080	Docker / Jenkins / AWS CLI / kubectl
Kubernetes Master	192.168.174.146 • v1.36.4	Control plane / API 6443
Worker 1 / Worker 2	192.168.174.147 / 192.168.174.148	Workload scheduling
AWS ECR	ap-southeast-2 • sample-ci-app	Private registry
Application	Namespace cicd-lab • NodePort 30081	Node.js sample-ci-app

1.3. Tiêu chí hoàn thành

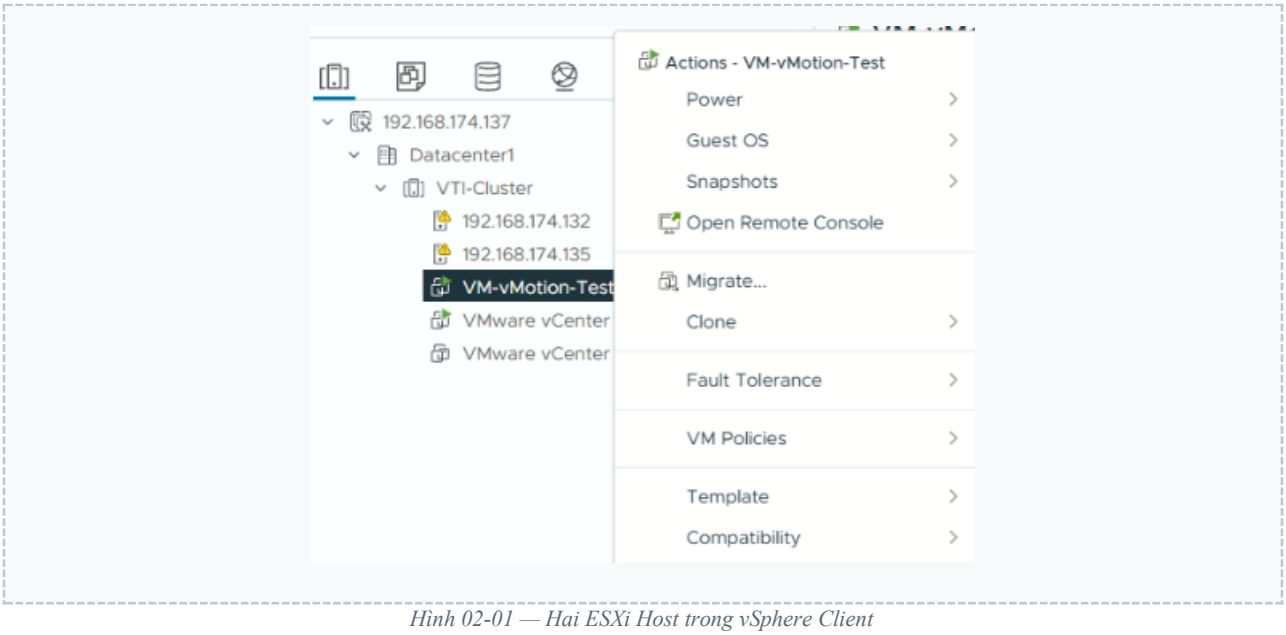
- GitHub Poll SCM kích hoạt Jenkins từ thay đổi trên nhánh main.
- Jenkins kiểm thử, build và push image có tag gắn với BUILD_NUMBER.
- Jenkins cập nhật k8s/deployment.yaml, commit có [skip ci] và deploy Kubernetes.
- Ứng dụng Running, rollout thành công và HTTP 200 qua NodePort 30081.

2. CHUẨN BỊ VMWARE ESXI VÀ NETWORK

2.1. Kiểm tra hạ tầng vSphere

Mục tiêu. Chuẩn bị hypervisor và mạng ảo để triển khai 3 VM Kubernetes. Hạ tầng đã được tạo trong những giai đoạn trước của lab.

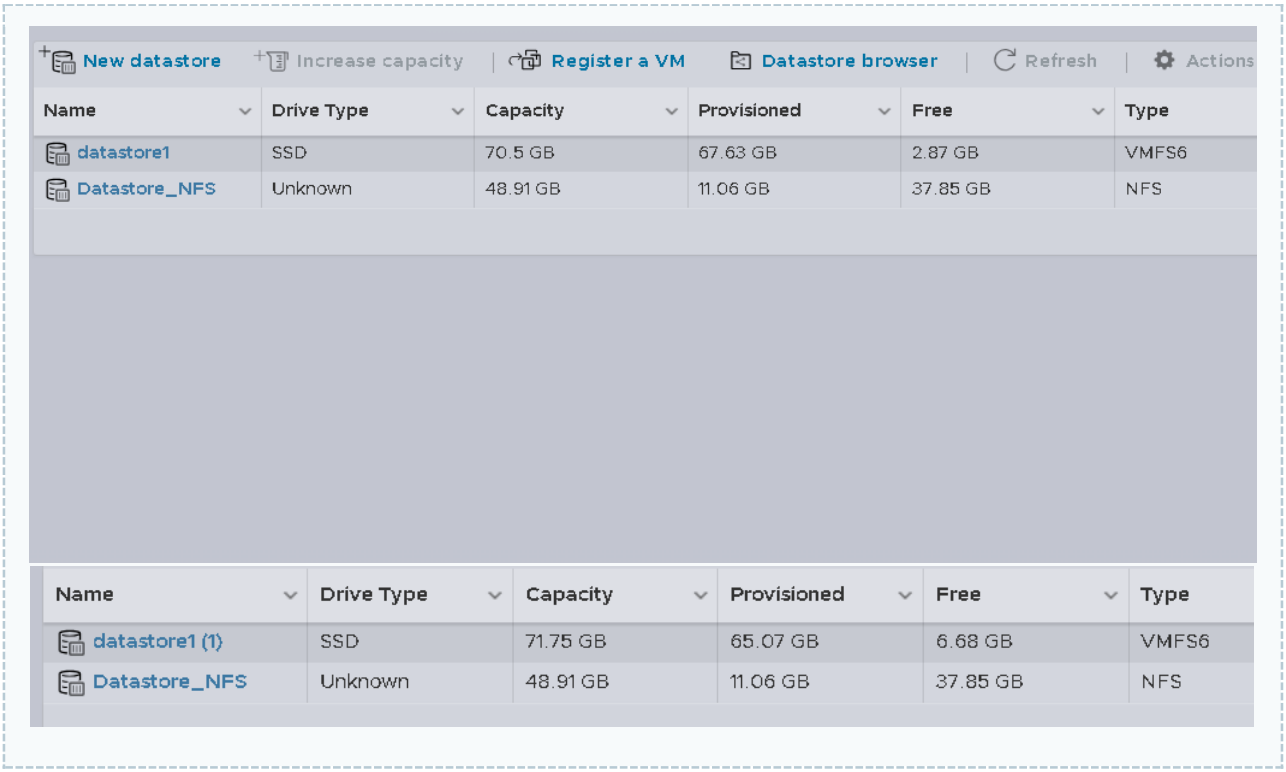
Tài nguyên	Thông tin ghi nhận
ESXi-01	192.168.174.132
ESXi-02	192.168.174.135
vCenter Server	192.168.174.137
Subnet cho Cluster Kubernetes	192.168.174.0/24
Jenkins VM / CI Subnet	192.168.232.170 / 192.168.232.0/24



Hình 02-01 — Hai ESXi Host trong vSphere Client

2.2. Dung lượng Datastore ESXi-01

Datastore	Dung lượng	Provisioned	Còn trống
datastore1 — VMFS6	70.50 GB	67.63 GB	2.87 GB (4.1%)
Datastore_NFS — NFS	48.91 GB	11.06 GB	37.85 GB (77.4%)

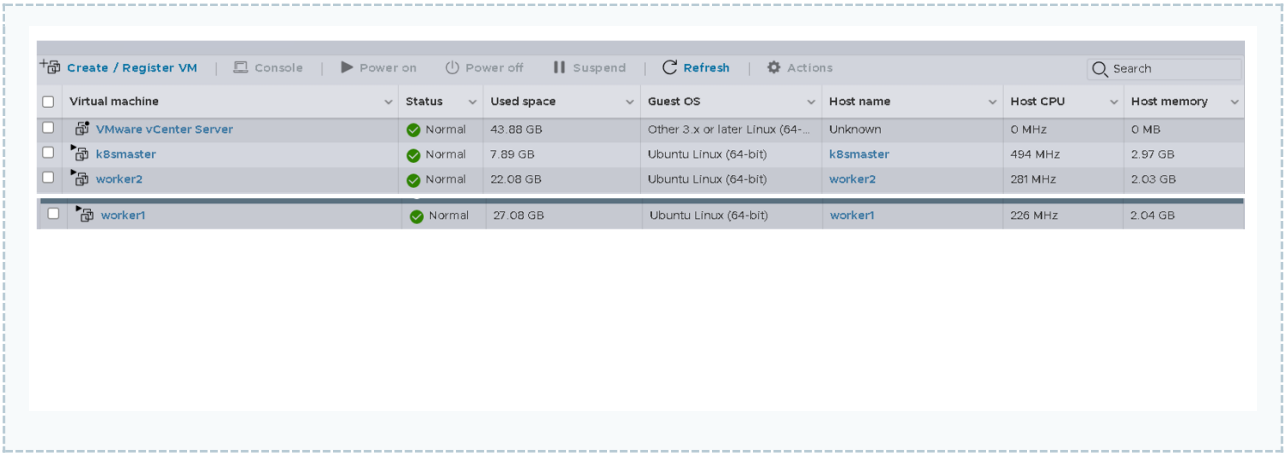


Hình 02-02 — Bảng Capacity / Provisioned / Free của hai Datastore

2.3. Máy ảo Ubuntu cho Kubernetes

Ba VM Ubuntu 24.04.4 LTS đã được cấu hình và vận hành.

VM	IP	Chức năng	Kết quả
k8smaster	192.168.174.146	Control Plane	Ready
worker1	192.168.174.147	Worker	Ready
worker2	192.168.174.148	Worker	Ready



Hình 02-03 — Danh sách ba Ubuntu VM trên ESXi

2.4. Kết nối mạng đến Kubernetes API

dk / Ubuntu — chỉ đọc, không thay đổi cấu hình

```
ip route get 192.168.174.146
curl -k -sS -o /dev/null -w 'HTTP %{http_code}\n' \
  --connect-timeout 5 https://192.168.174.146:6443/version
```

Kết quả đã kiểm chứng. Từ dk, Kubernetes API trả HTTP 200 tại 192.168.174.146:6443. Mạng gửi qua gateway 192.168.232.2. Không yêu cầu Jenkins dùng Cloudflare làm tuyến bắt buộc.

3. CÀI ĐẶT MÁY CI-LAB: DOCKER VÀ JENKINS

3.1. Chuẩn bị CI Server dk

Mục tiêu. Sử dụng Ubuntu VM trên Laptop làm CI Server, chạy Jenkins trong Docker, tích hợp Docker Engine, AWS CLI và kubectl.

Thông số	Cấu hình thực tế
Hostname / địa chỉ	dk — 192.168.232.170
Jenkins Container	jenkins / image jenkins-with-docker:lts
Jenkins Web	http://192.168.232.170:8080
Workspace Git (host)	/home/dk/jenkins-ci-lab/sample-app
Jenkins Home (host)	/home/dk/jenkins-ci-lab/jenkins_home
Jenkins Job	sample-ci-pipeline

3.2. Kiểm tra Docker Engine và Jenkins Container

dk / Ubuntu Terminal — lệnh kiểm tra

```
docker --version
docker ps --filter name=jenkins
docker exec jenkins sh -c 'docker --version; aws --version; kubectl version --client'
```

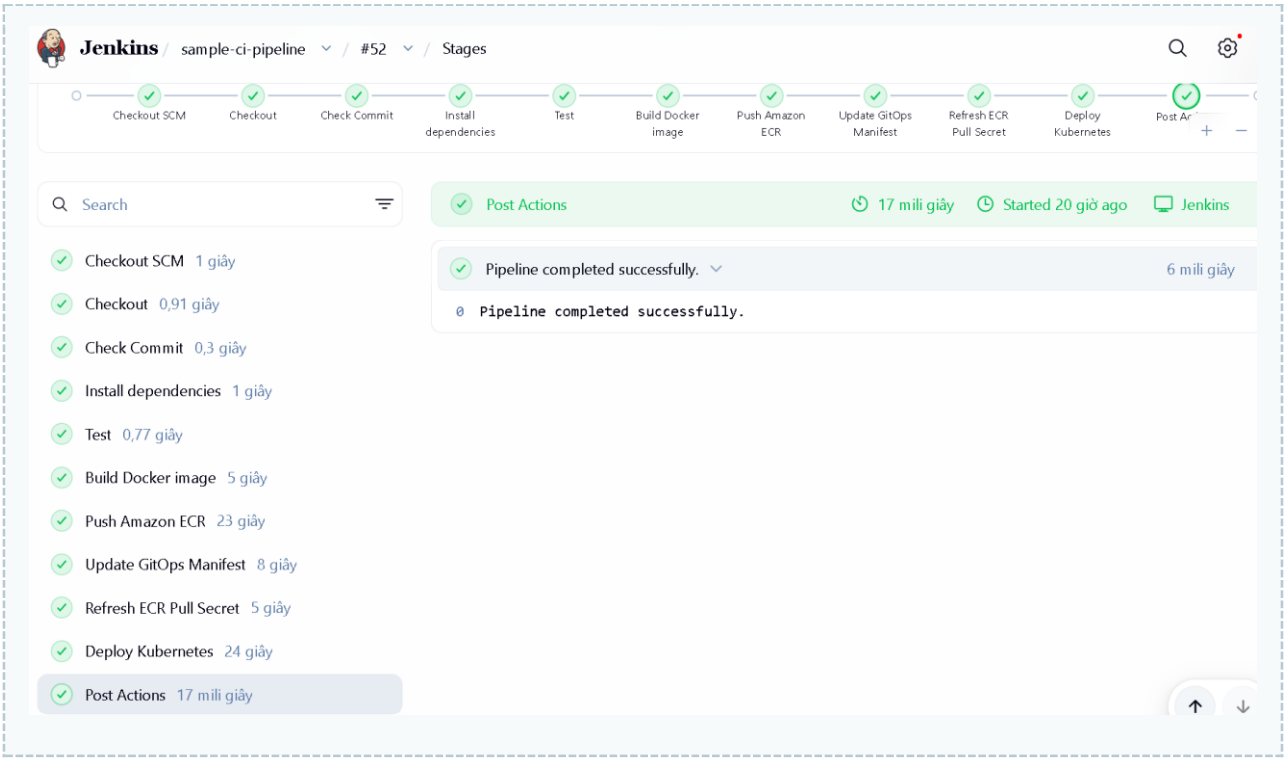
Kết quả. Jenkins container đã hoạt động trên port 8080 và 50000 trong các lần kiểm tra trước; Jenkins sử dụng Docker daemon của host để build/push image.

```
dk@dk:~$ docker --version
docker ps --filter name=jenkins
docker exec jenkins sh -c 'docker --version; aws --version; kubectl version --client'
Docker version 29.8.0, build 88096ef
CONTAINER ID   IMAGE                    COMMAND                  CREATED      STATUS       PORTS
                         NAMES
bf403feac59c   jenkins-with-docker:lts   "/usr/bin/tini -- /u…"   7 days ago   Up 2 hours   0.0.0.0:8080->8080/tcp
0/tcp, [::]:50000->50000/tcp   jenkins
Docker version 26.1.5+dfsg1, build a72d7cd
aws-cli/2.37.9 Python/3.14.6 Linux/6.8.0-146-generic exe/x86_64.debian.13
Client Version: v1.36.4
Kustomize Version: v5.8.1
```

Hình 03-01 — Container Jenkins đang Up, port 8080/50000

3.3. Jenkins Web UI và Job

Truy cập Jenkins Dashboard, xác nhận job sample-ci-pipeline có Build History và trạng thái Build #52 SUCCESS.



Hình 03-02 — Jenkins Dashboard và job sample-ci-pipeline

3.4. Các công cụ CI trong Jenkins

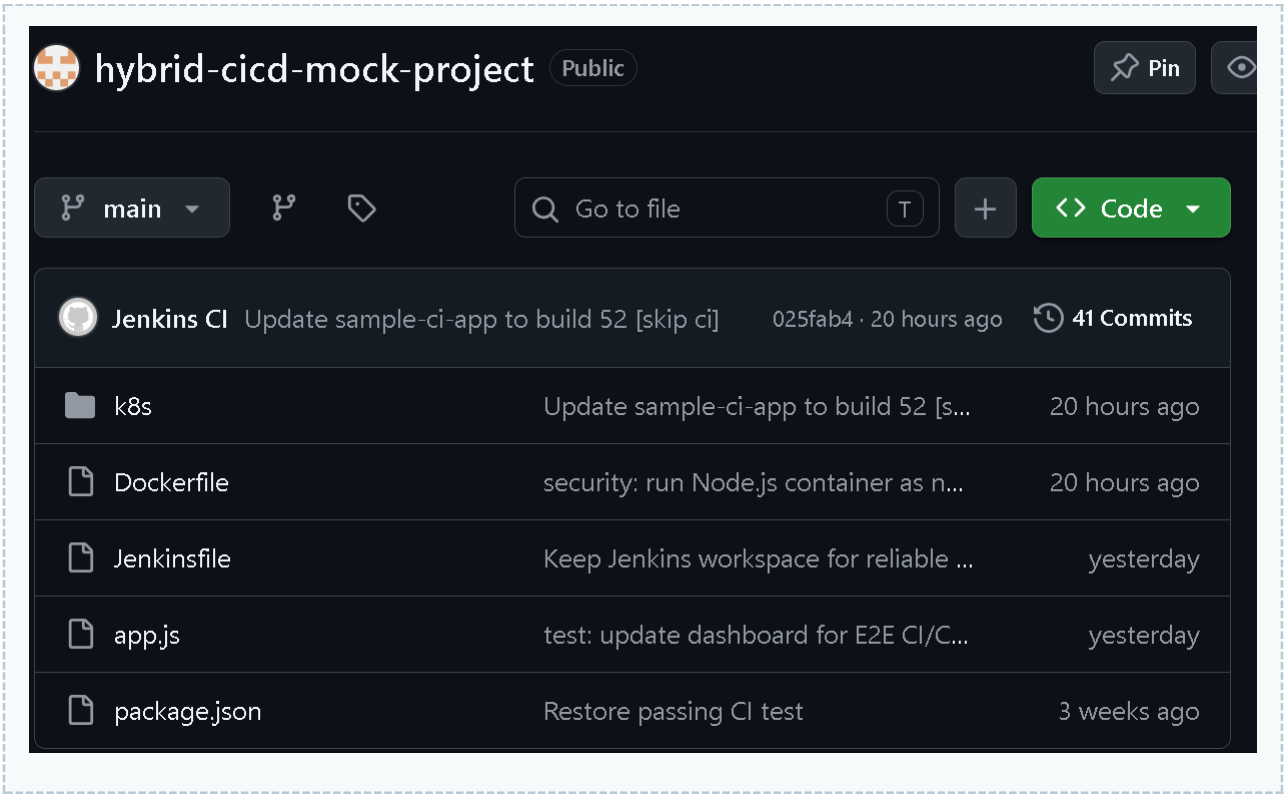
Jenkins thực hiện checkout Git, chạy lệnh Node.js/npm trong container node:20-alpine, Docker Build, AWS CLI push ECR và kubectl deploy. Credential kubeconfig chỉ được cấp vào các stage Kubernetes thông qua withCredentials().

4. CHUẨN BỊ MÃ NGUỒN GITHUB VÀ AWS ECR

4.1. Repository và cấu trúc thư mục

Repository. https://github.com/hungdoan42304/hybrid-cicd-mock-project — nhánh main.

```
hybrid-cicd-mock-project/
├── app.js
├── package.json
├── Dockerfile
├── Jenkinsfile
└── k8s/
    ├── deployment.yaml
    └── service.yaml
```



Hình 04-01 — GitHub repository, branch main, Dockerfile / Jenkinsfile / k8s

4.2. Ứng dụng Node.js mẫu

Ứng dụng Express chạy port 3000, đọc biến môi trường APP_VERSION (từ BUILD_NUMBER) và hiển thị trạng thái Hybrid CI/CD Dashboard. package.json cung cấp lệnh npm start và npm test.

GitHub / app.js — trích đoạn (không phải toàn bộ mã nguồn)

```
const port = process.env.PORT || 3000;
const version = process.env.APP_VERSION || "local";
// Dashboard hiển thị số build và registry Amazon ECR.
app.listen(port, () => {
  console.log(`Dashboard version ${version} running on port ${port}`);
});
```

4.3. Dockerfile sử dụng non-root

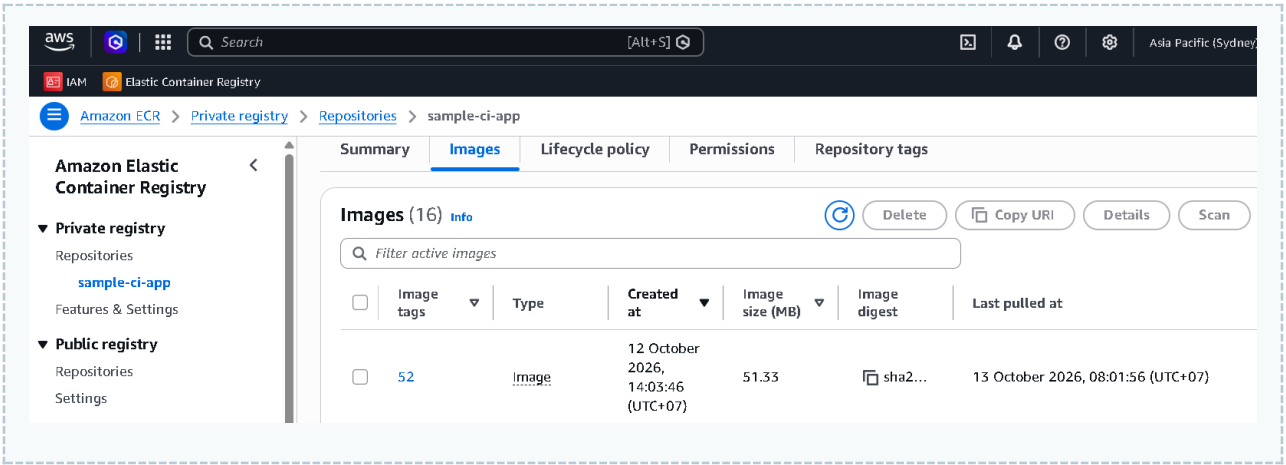
GitHub / Dockerfile — nguyên nội dung tại commit 025fab4

```
FROM node:20-alpine
WORKDIR /app
ARG BUILD_NUMBER=local
ENV APP_VERSION=${BUILD_NUMBER}
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
RUN chown -R node:node /app
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

Ý nghĩa. Docker Image sử dụng node:20-alpine; APP_VERSION phản ánh Build Number; ứng dụng chạy dưới user node (UID 1000), không phải root.

4.4. Amazon ECR Private Registry

Thuộc tính	Giá trị
AWS Region	ap-southeast-2
Repository	sample-ci-app
Image triển khai	510869689558.dkr.ecr.ap-southeast-2.amazonaws.com/sample-ci-app:52
IAM cho Jenkins	mockproject-jenkins / policy JenkinsECRSampleCIAppPolicy



Hình 04-02 — ECR repository sample-ci-app và tag 52

4.5. Kiểm tra quyền truy cập ECR không hiển thị secret

Jenkins/dk đã cấu hình AWS CLI phù hợp — không in khóa bí mật

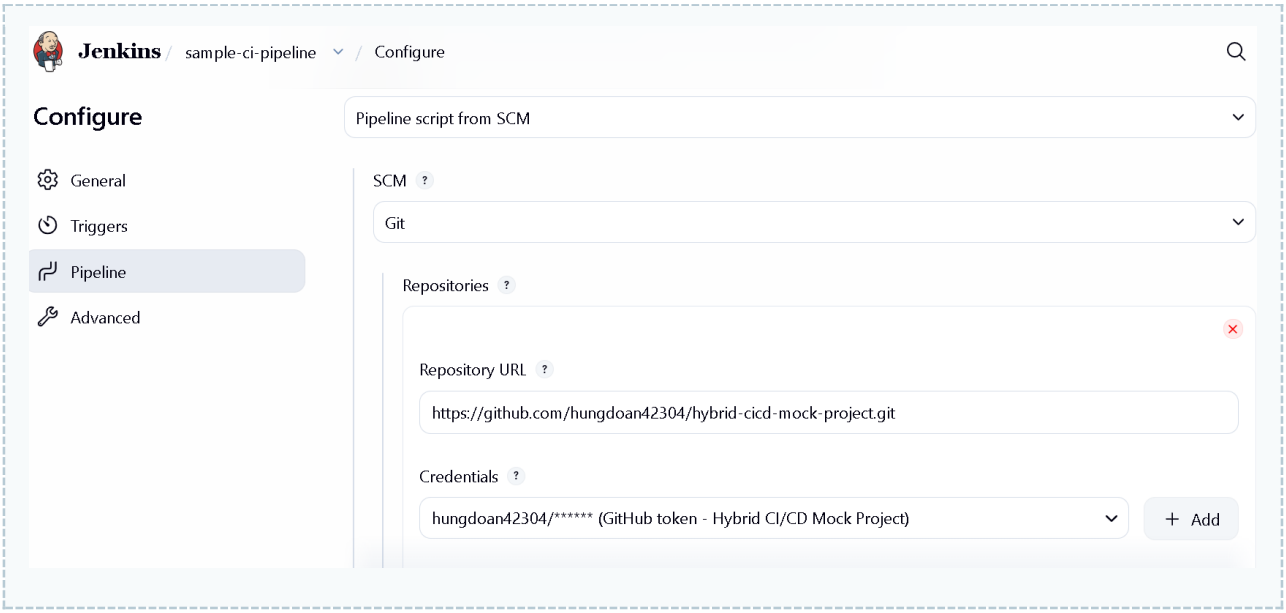
```
aws ecr describe-repositories \
  --repository-names sample-ci-app \
  --region ap-southeast-2 \
  --query 'repositories[0].{Name:repositoryName,URI:repositoryUri}'

aws ecr list-images \
  --repository-name sample-ci-app \
  --region ap-southeast-2
```

5. CẤU HÌNH JENKINS PIPELINE VÀ GITHUB POLL SCM

5.1. Tạo Pipeline Job dùng Jenkinsfile từ GitHub

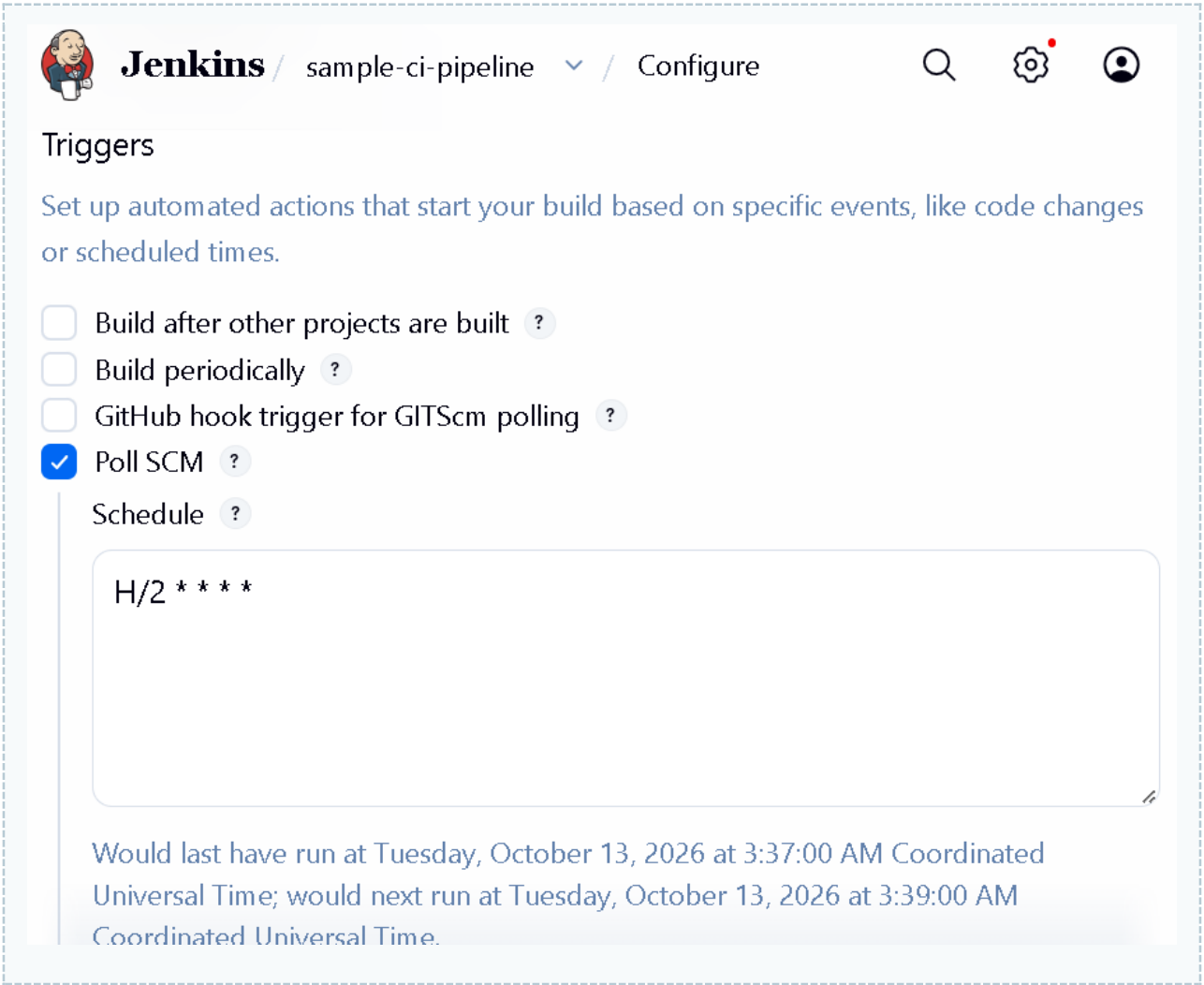
Cấu hình. Jenkins job sample-ci-pipeline, Definition: Pipeline script from SCM, SCM: Git, Branch: main, Script Path: Jenkinsfile.



Hình 05-01 — Job Configure: Pipeline script from SCM / Git Repository

5.2. Trigger Poll SCM

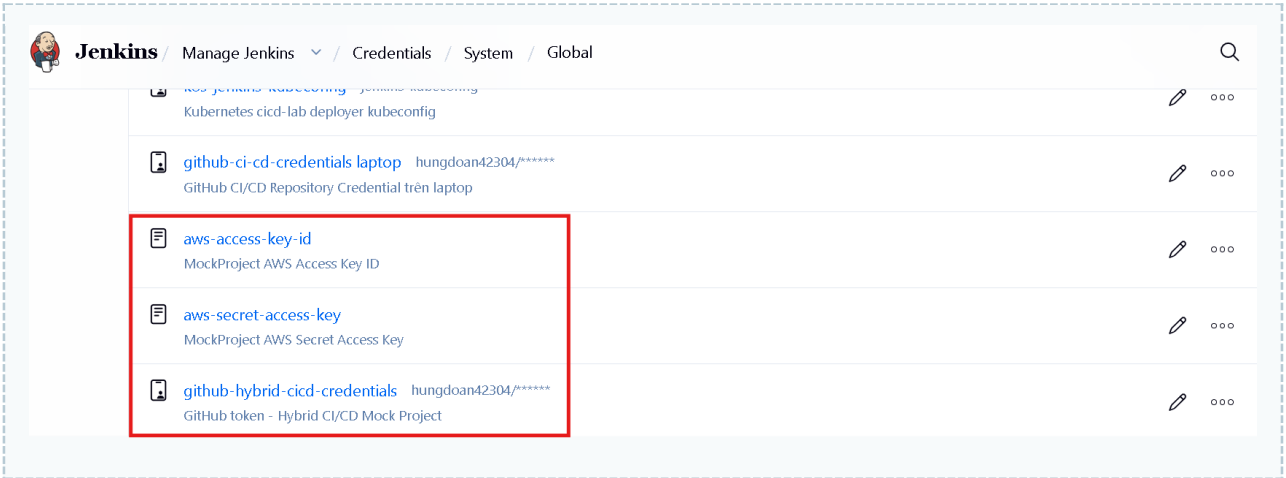
Lịch kiểm tra. H/2 * * * * — Jenkins kiểm tra SCM khoảng mỗi 2 phút; lab không bắt buộc GitHub Webhook phải gọi ngược về Jenkins.



*Hình 05-02 — Build Triggers: Poll SCM và lịch H/2 * * * *

5.3. Jenkins Credentials

Credentials ID	Kind / mục đích
aws-access-key-id	Secret text / Access Key ID
aws-secret-access-key	Secret text / Secret Access Key
k8s-vmware-deployer	Secret File / jenkins-kubeconfig



Hình 05-03 — Danh sách Jenkins Credentials

5.4. Các stage chính trong Jenkinsfile

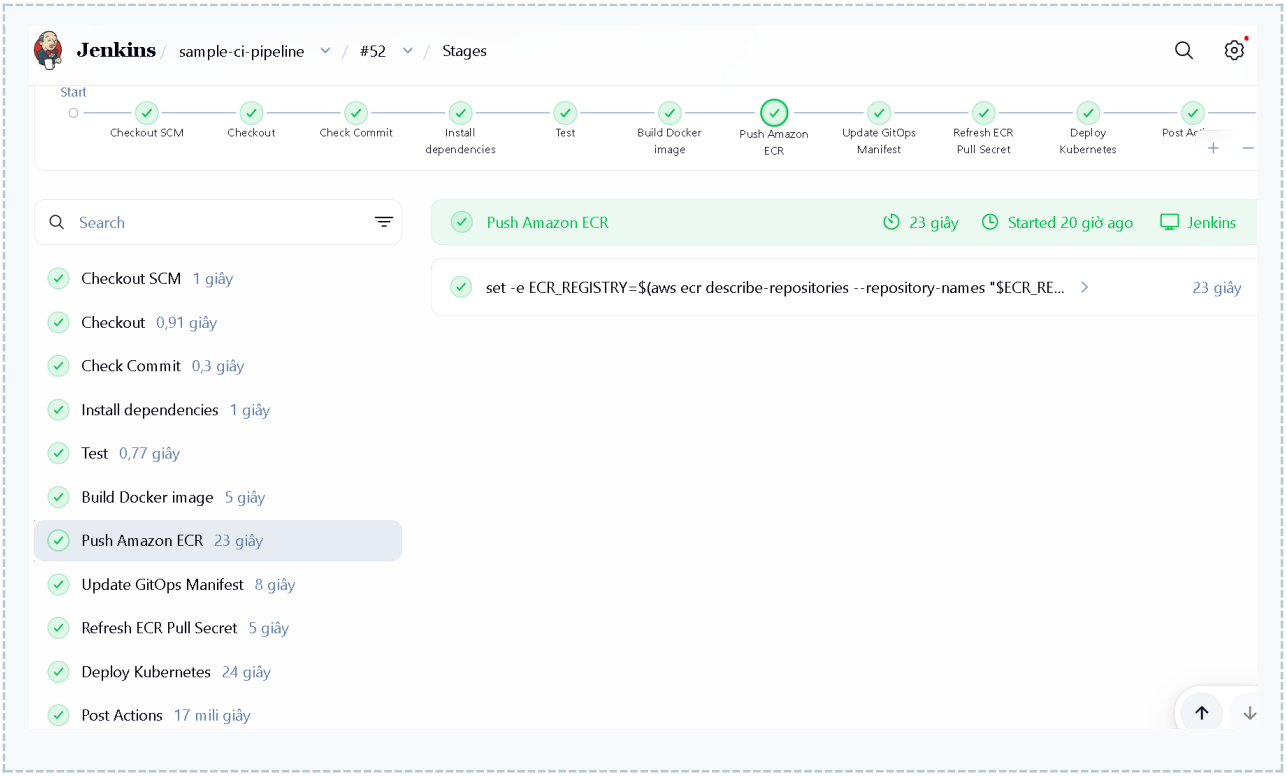
Stage	Chức năng
Checkout	Lấy source code từ GitHub / main
Check Commit	Nếu tác giả commit là Jenkins CI → SKIP_CI=true
Install dependencies + Test	node:20-alpine chạy npm install, npm test
Build Docker image	Tạo sample-ci-app:$BUILD_NUMBER
Push Amazon ECR	Docker login qua AWS CLI, tag và push image
Update GitOps Manifest	Đổi image trong k8s/deployment.yaml, commit [skip ci]
Refresh ECR Pull Secret	Cập nhật secret ecr-pull-secret qua kubectl
Deploy Kubernetes	kubectl apply, rollout status và get pods

5.5. Ngăn vòng lặp CI bởi GitOps commit

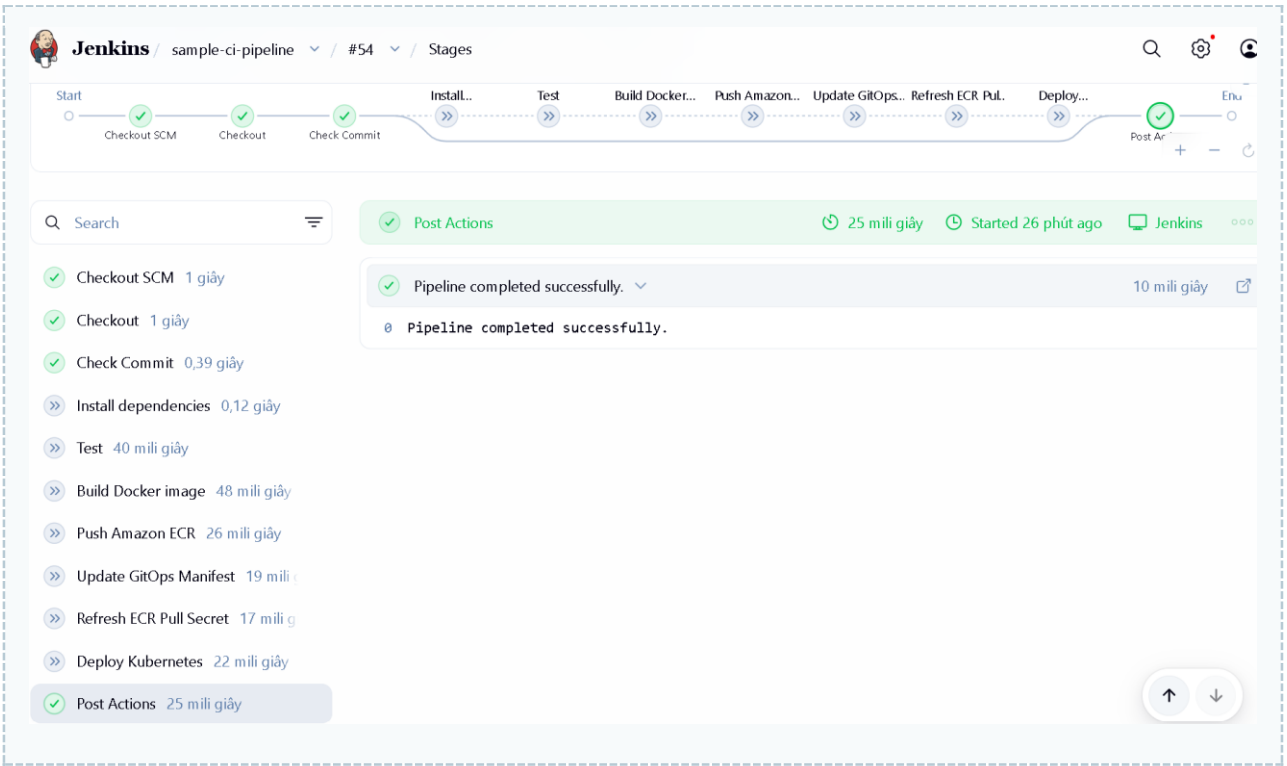
Jenkinsfile / Check Commit — trích đoạn chính xác

```
if (env.LAST_COMMIT_AUTHOR == 'Jenkins CI') {
    env.SKIP_CI = 'true'
    echo 'Commit was created by Jenkins CI. Skipping CI stages.'
} else {
    env.SKIP_CI = 'false'
}
```

Bằng chứng. Build #52 đã chạy đầy đủ các stage CI/CD. Jenkins commit 025fab4 (Update sample-ci-app to build 52 [skip ci]); Build #54 SUCCESS và bỏ qua các stage CI/CD theo cơ chế chống vòng lặp.



Hình 05-04 — Build #52 SUCCESS và Pipeline Stage View



Hình 05-05 — Build #54 nhận commit do Jenkins CI tạo và skip các stage

6. TRIỂN KHAI KUBERNETES CLUSTER TRÊN VMWARE

6.1. Cấu trúc Cluster

Mục tiêu. Vận hành Kubernetes 1 Control Plane + 2 Worker trên 3 VM Ubuntu thuộc VMware ESXi.

Node	Role	IP	Hệ điều hành / Runtime
k8smaster	control-plane	192.168.174.146	Ubuntu 24.04.4 / containerd 2.3.5
worker1	worker	192.168.174.147	Ubuntu 24.04.4 / containerd 2.3.5
worker2	worker	192.168.174.148	Ubuntu 24.04.4 / containerd 2.3.5

6.2. Xác minh Nodes / System Pods

k8s@k8smaster — sử dụng kubeconfig hợp lệ

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get namespace cicd-lab
```

Kết quả. Kubernetes v1.36.4; cả k8smaster, worker1, worker2 đều Ready. Trước đó đã kiểm chứng worker1 được tạo lại sau failover.

```
k8s@k8smaster:~$ kubectl get nodes -o wide
NAME         STATUS   ROLES           AGE    VERSION   INTERNAL-IP
k8smaster    Ready    control-plane   4d2h   v1.36.4   192.168.174.146
worker1      Ready    <none>          4d2h   v1.36.4   192.168.174.147
worker2      Ready    <none>          4d2h   v1.36.4   192.168.174.148
```

Hình 06-01 — kubectl get nodes -o wide — 3/3 Ready

```
k8s@k8smaster:~$ kubectl get pods -n kube-system
kubectl get namespace cicd-lab
NAME                                       READY   STATUS    RESTARTS
AGE
calico-kube-controllers-84bbd545cc-p6pzv   1/1     Running   8 (175m ago)
4d2h
calico-node-82jxj                          1/1     Running   5 (175m ago)
4d2h
calico-node-d96bj                          1/1     Running   5 (175m ago)
4d2h
calico-node-xh28z                          1/1     Running   4 (176m ago)
4d2h
coredns-db7c7cbf8-fqkdc                    1/1     Running   4 (176m ago)
4d2h
coredns-db7c7cbf8-l2xbp                    1/1     Running   5 (175m ago)
4d2h
dns-autoscaler-68bc5c675d-xf8zs            1/1     Running   5 (175m ago)
4d2h
kube-apiserver-k8smaster                   1/1     Running   6 (175m ago)
4d2h
kube-controller-manager-k8smaster          1/1     Running   7 (175m ago)
4d2h
kube-proxy-5j82s                           1/1     Running   5 (175m ago)
4d2h
kube-proxy-7bpdf                           1/1     Running   5 (175m ago)
4d2h
kube-proxy-pllhj                           1/1     Running   4 (176m ago)
4d2h
kube-scheduler-k8smaster                   1/1     Running   8 (175m ago)
4d2h
metrics-server-765c774d7b-skbhz            1/1     Running   4 (176m ago)
4d2h
nodelocaldns-5xvhh                         1/1     Running   4 (175m ago)
3d19h
nodelocaldns-kbvrb                         1/1     Running   4 (175m ago)
3d19h
nodelocaldns-zrx7k                         1/1     Running   6 (176m ago)
3d19h
NAME       STATUS   AGE
cicd-lab   Active   4d
```

Hình 06-02 — Các Pod hệ thống và namespace cicd-lab

6.3. Mạng Kubernetes và DNS nội bộ

Cluster hoạt động với containerd; DNS nội bộ được kiểm thử lại thành công bằng nslookup kubernetes.default.svc.cluster.local trên Pod thuộc worker2. Test Pod tạm đã được xóa.

k8smaster — chỉ kiểm tra

```
kubectl get svc kubernetes -n default
kubectl get svc -n kube-system
kubectl get pods -n kube-system -o wide
```

6.4. Tài nguyên và trạng thái Node

k8smaster — kiểm tra sử dụng RAM và node conditions

```
kubectl top nodes
kubectl describe node worker1
```

Kết quả. Worker1 từng có memory usage 109% so với Allocatable nhưng MemoryPressure=False và Ready=True; không được đánh đồng mức vượt Allocatable với việc hết RAM vật lý.

7. KUBERNETES MANIFESTS VÀ JENKINS DEPLOYMENT

7.1. Namespace và tài nguyên ứng dụng

Namespace. cicd-lab. Deployment và Service cùng tên sample-ci-app. Image lấy từ Amazon ECR private.

k8smaster / Terminal

```
kubectl get deployment,svc,pods,cronjobs -n cicd-lab -o wide
```

```
k8s@k8smaster:~$ kubectl get deployment,svc,pods,cronjobs -n cicd-lab
kubectl get pods -n cicd-lab -o wide | grep sample-ci-app
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/sample-ci-app       1/1     1            1           3d23h

NAME                     TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/sample-ci-app    NodePort   10.233.20.37    <none>        3000:30081/TCP   3d19h

NAME                                    READY   STATUS      RESTARTS       AGE
pod/dns-check                           0/1     Completed   0              3d20h
pod/dns-test-worker2                    0/1     Completed   0              3d20h
pod/ecr-recovery-20261013010137-tnnt9   0/1     Completed   0              3h4m
pod/ecr-token-refresh-29864160-44v8l    0/1     Error       0              3h4m
pod/ecr-token-refresh-29864160-v8d6q    0/1     Completed   0              3h4m
pod/sample-ci-app-f44f99866-c9qt6       1/1     Running     3 (3h5m ago)   20h

NAME                                SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/ecr-token-refresh     0 */6 * * *   <none>     False     0        4h7m            3d20h
sample-ci-app-f44f99866-c9qt6        1/1     Running     3 (3h5m ago)   20h     10.233.125.40    worker2
```

Hình 07-01 — Deployment 1/1; Service 3000:30081; Pod Running trên worker2

7.2. Deployment YAML — các phần quan trọng

GitHub / k8s/deployment.yaml — trích cấu hình thực tế

```
spec:
  replicas: 1
  template:
    spec:
      imagePullSecrets:
        - name: ecr-pull-secret
      containers:
        - name: sample-ci-app
          image: 510869689558.dkr.ecr.ap-southeast-2.amazonaws.com/sample-ci-app:52
          imagePullPolicy: Always
          securityContext:
            runAsNonRoot: true
            runAsUser: 1000
            runAsGroup: 1000
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
```

Manifest đầy đủ còn có nodeAffinity ưu tiên worker có nhãn workload-priority=primary, readinessProbe, livenessProbe và requests/limits.

7.3. Service NodePort 30081

GitHub / k8s/service.yaml — nguyên nội dung

```
apiVersion: v1
kind: Service
metadata:
  name: sample-ci-app
  namespace: cicd-lab
spec:
  type: NodePort
  selector:
    app: sample-ci-app
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30081
```

```
k8s@k8smaster:~$ kubectl get svc sample-ci-app -n cicd-lab
NAME            TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
sample-ci-app   NodePort   10.233.20.37   <none>        3000:30081/TCP   3d19h
k8s@k8smaster:~$ curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" http://192.168.174.148:30081
HTTP Status: 200
```

Hình 07-02 — Service NodePort 30081 và ứng dụng trả HTTP 200

7.4. ECR Pull Secret và CronJob làm mới token

Cấu hình. Secret ecr-pull-secret, loại kubernetes.io/dockerconfigjson; Deployment tham chiếu imagePullSecrets. CronJob ecr-token-refresh chạy lịch 0 */6 * * *, SUSPEND=False.

k8smaster — không xuất nội dung secret

```
kubectl get secret ecr-pull-secret -n cicd-lab \
  -o jsonpath='{.type}{"\n"}'
kubectl get cronjob ecr-token-refresh -n cicd-lab
kubectl get jobs -n cicd-lab
```

Kết quả. Job refresh đã Complete, log xác nhận secret/ecr-pull-secret replaced và ECR Pull Secret refreshed successfully; thử pull image với imagePullPolicy=Always thành công.

```
k8s@k8smaster:~$ kubectl get secret ecr-pull-secret -n cicd-lab \
  -o jsonpath='{.type}{"\n"}'
kubernetes.io/dockerconfigjson
k8s@k8smaster:~$ kubectl get cronjob ecr-token-refresh -n cicd-lab
kubectl get jobs -n cicd-lab
NAME                SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE
ecr-token-refresh   0 */6 * * *   <none>     False     0        4h11m
NAME                           STATUS     COMPLETIONS   DURATION   AGE
ecr-recovery-20261013010137    Complete   1/1           13s        3h9m
ecr-token-refresh-29864160     Complete   1/1           32s        3h9m
```

Hình 07-03 — CronJob ecr-token-refresh và Completed Job

7.5. Deployment Strategy và Rollout

k8smaster

```
kubectl rollout status deployment/sample-ci-app \
  -n cicd-lab --timeout=60s
kubectl get deployment sample-ci-app -n cicd-lab \
  -o jsonpath='{.spec.replicas}{" replicas\n"}{.spec.strategy.type}{"\n"}'
```

Kết quả. replicas=1, RollingUpdate; maxSurge=25% và maxUnavailable=25%. Với 1 replica: surge làm tròn lên 1, unavailable làm tròn xuống 0. Rollout đã báo successfully rolled out.

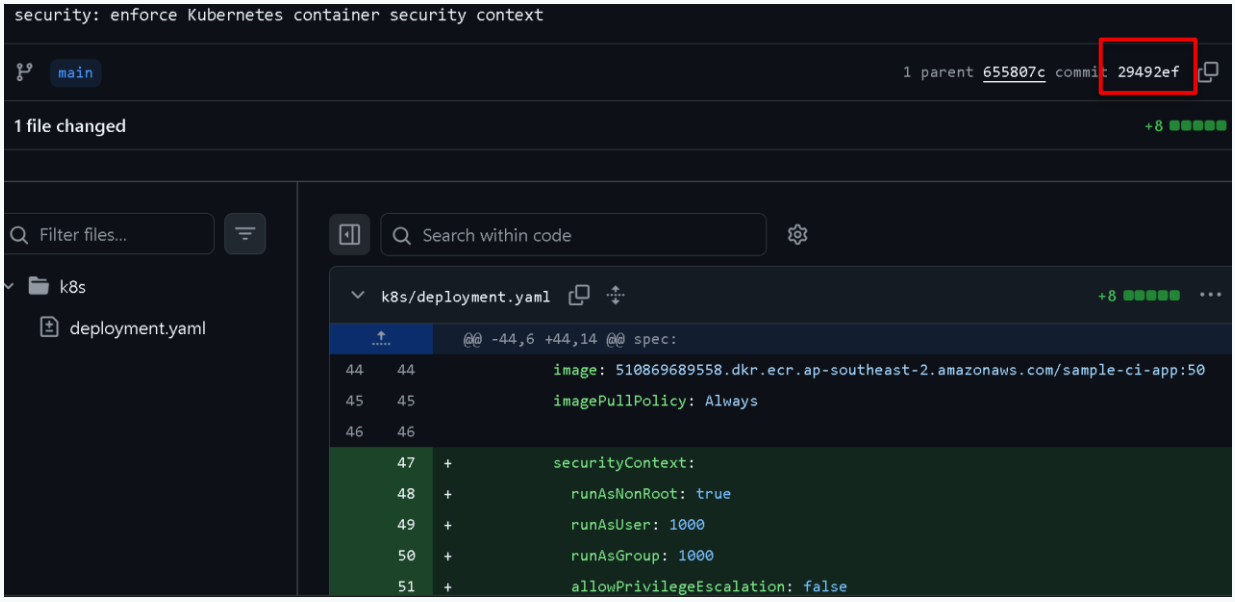
8. KIỂM THỬ TOÀN BỘ LUỒNG HYBRID CI/CD

8.1. Kịch bản End-to-End

Kiểm chứng một thay đổi ứng dụng đẩy lên GitHub main được Jenkins Poll SCM nhận diện, tự động chạy Test → Build → ECR Push → GitOps Commit → Kubernetes Deployment; không build hoặc deploy thủ công từng bước.

dk / ~/jenkins-ci-lab/sample-app — kiểm tra trạng thái hiện có

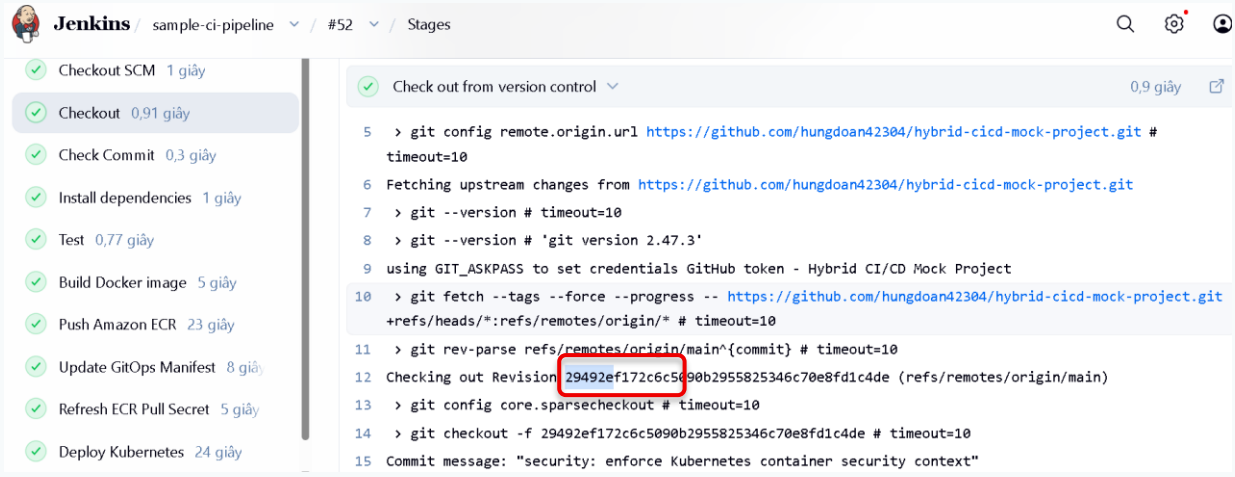
```
git status -sb
git log -1 --oneline origin/main
# Khi có thay đổi source thực sự mới: git add / commit / push
# Đã nghiệm thu trên Build #52; không yêu cầu push thêm để lấy ảnh.
```



Hình 08-01 — GitHub source commit kích hoạt Jenkins CI

8.2. Quan sát Jenkins Build #52

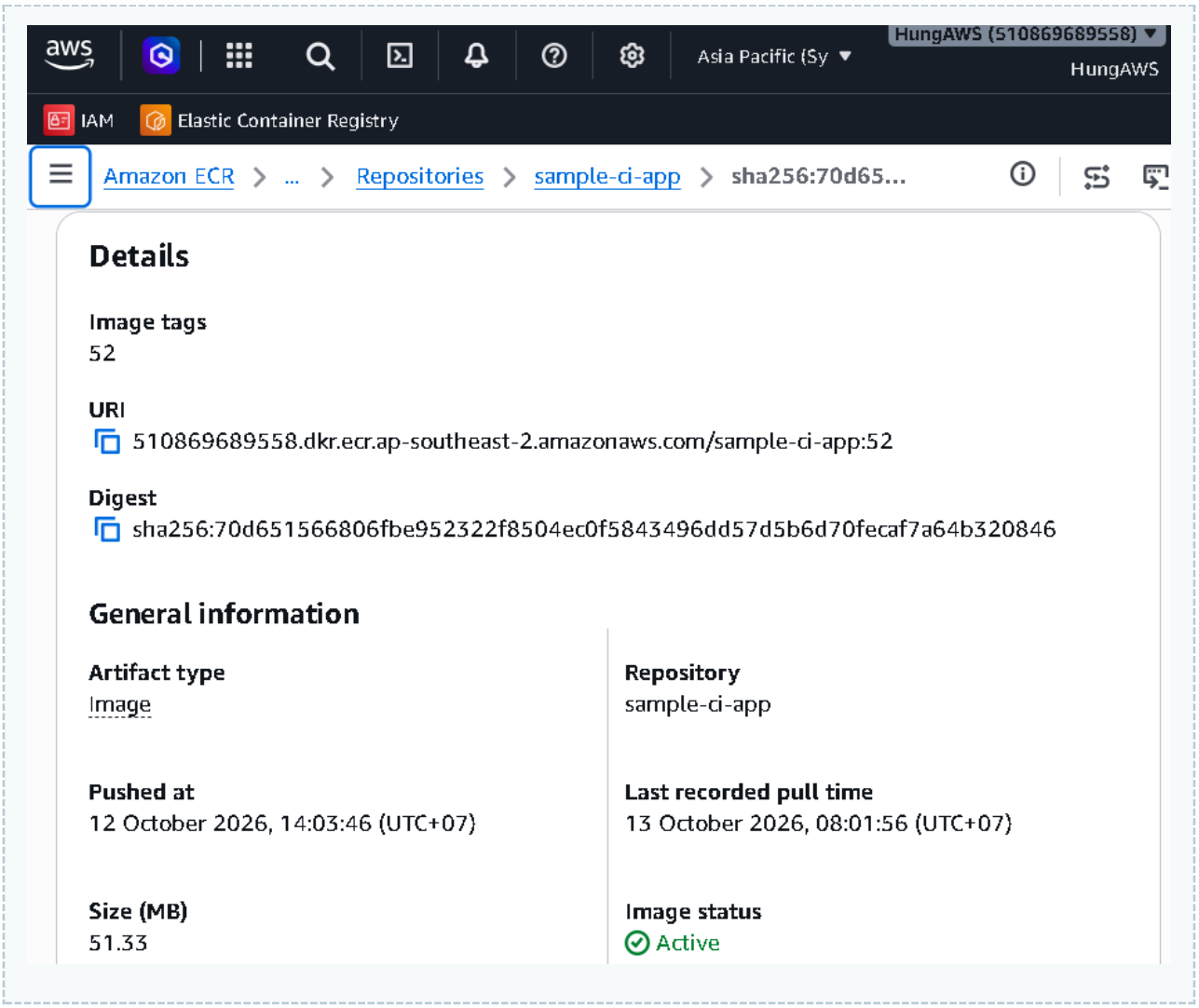
Bằng chứng xác minh. Build #52 SUCCESS: npm test, Docker Build chạy non-root, ECR Push và Kubernetes Deploy thành công.



Hình 08-02 — Jenkins Build #52 / Stages và Console Output PASS

8.3. Quan sát Amazon ECR

Bằng chứng xác minh. Image sample-ci-app:52 đã push thành công; digest sha256:70d651566806fbe952322f8504ec0f5843496dd57d5b6d70fecaf7a64b320846.



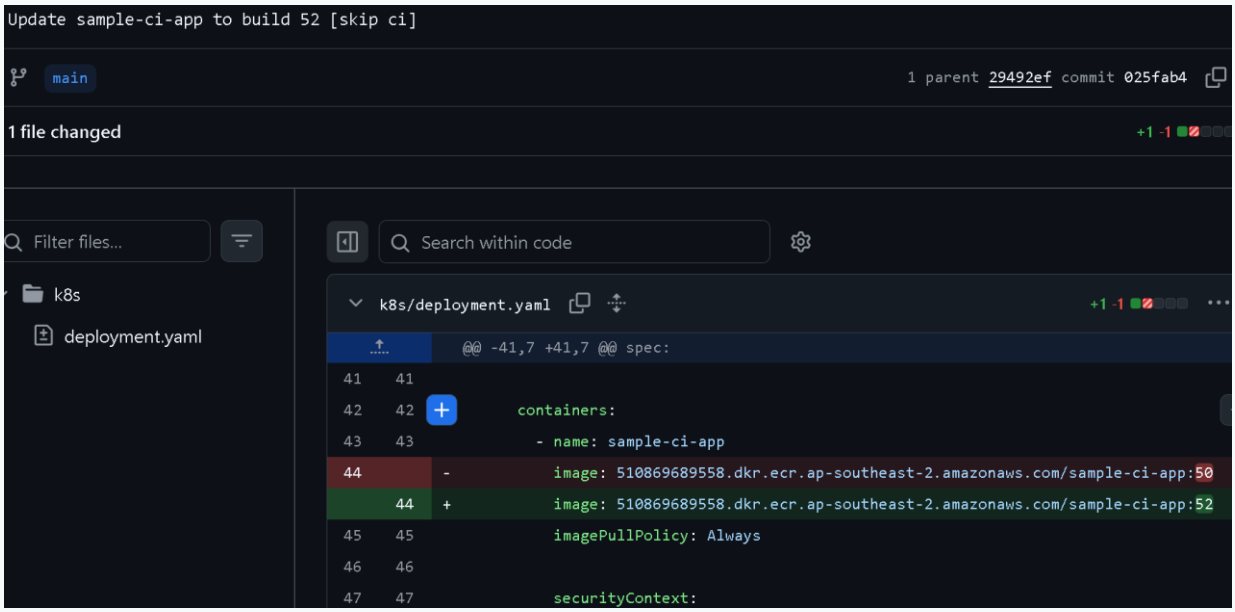
Hình 08-03 — Amazon ECR — image tag 52 / digest

8.4. Quan sát GitOps commit và khả năng chống lặp

Bằng chứng xác minh. Jenkins tạo commit 025fab4 với message Update sample-ci-app to build 52 [skip ci]. Build #53 nhận commit Jenkins CI và bỏ qua các stage theo thiết kế.

dk / Git Repository — read-only

```
git show --stat --oneline 025fab4
grep 'image:' k8s/deployment.yaml
```



Hình 08-04 — GitHub commit 025fab4 thay đổi image :52

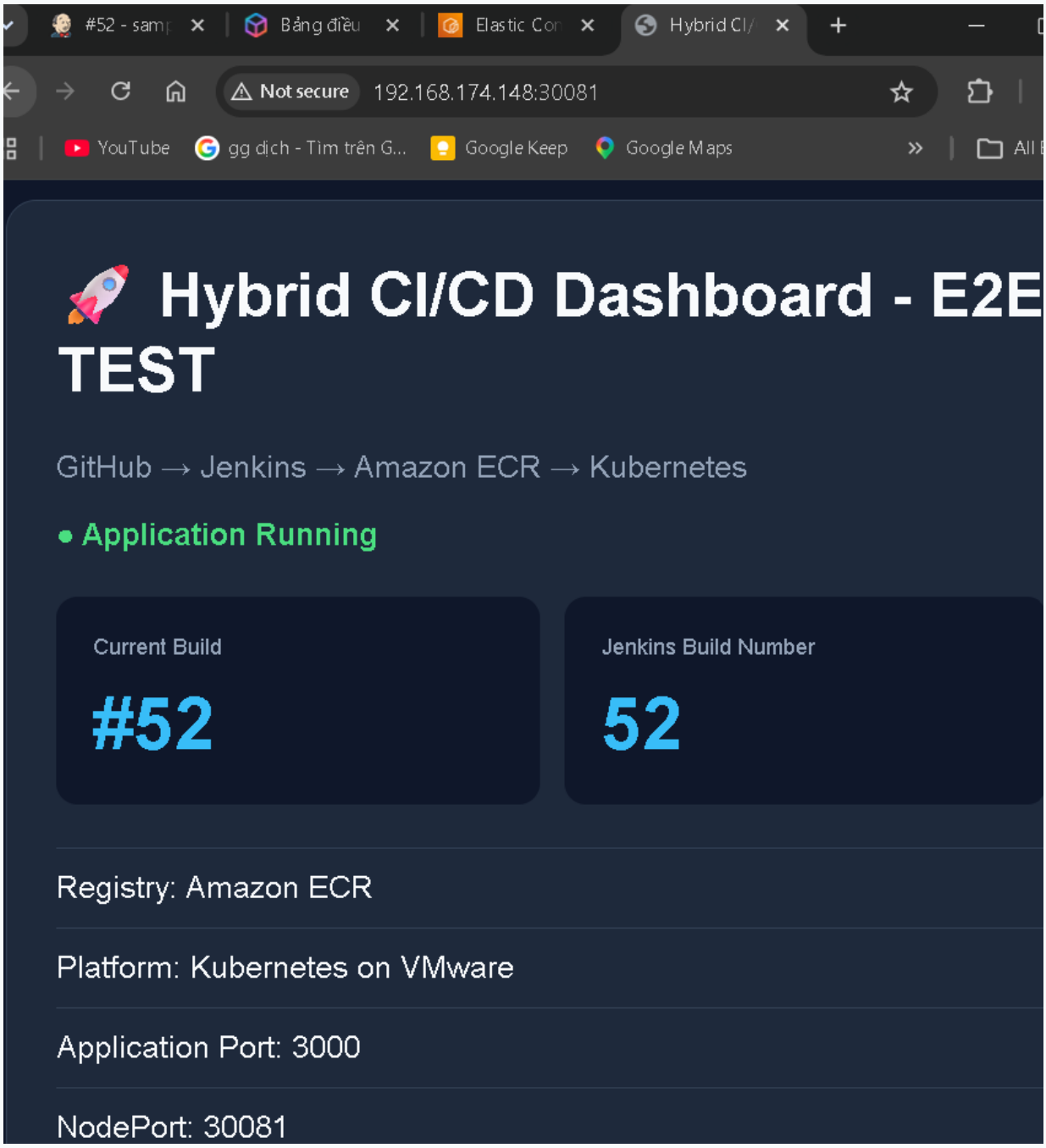
8.5. Quan sát Kubernetes rollout và HTTP

k8smaster

```
kubectl rollout status deployment/sample-ci-app -n cicd-lab
kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
```

dk / Ubuntu — endpoint đã kiểm chứng HTTP 200

```
curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
  http://192.168.174.148:30081/
```



Hình 08-05 — Application Dashboard hiển thị Build #52 qua NodePort 30081

8.6. Bảng nghiệm thu End-to-End

Thành phần	Kết quả	Bằng chứng
GitHub → Jenkins Poll SCM	PASS	Build #52 được kích hoạt từ SCM
npm Test + Docker Build	PASS	Jenkins Build #52 SUCCESS
ECR Push / Private Image Pull	PASS	Tag :52, test pull Always
GitOps commit	PASS	Commit 025fab4 / [skip ci]

Thành phần	Kết quả	Bằng chứng
Kubernetes Deploy	PASS	Rollout thành công, Pod Running
NodePort 30081	PASS	HTTP 200
Full Infrastructure Restart	CHƯA TEST	Giữ lại bài Recovery Test

9. VẬN HÀNH, BẢO MẬT VÀ XỬ LÝ LỖI THƯỜNG GẶP

9.1. Pod Self-healing

Kịch bản đã kiểm thử. Xóa Pod ứng dụng có kiểm soát. Deployment tự tạo Pod thay thế; endpoint phục vụ HTTP 200 sau khi Pod sẵn sàng. Không cần lặp lại thao tác xóa Pod chỉ để chụp ảnh.

k8smaster — chỉ kiểm tra trạng thái hiện tại

```
kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
kubectl get deployment sample-ci-app -n cicd-lab
```

```
k8s@k8smaster:~$ kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
kubectl get deployment sample-ci-app -n cicd-lab
NAME                              READY   STATUS    RESTARTS        AGE   IP              NODE
sample-ci-app-f44f99866-c9qt6   1/1     Running   3 (5h26m ago)   22h   10.233.125.40   worker2
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
sample-ci-app   1/1     1            1           4d2h
```

Hình 09-01 — Pod được tái tạo / Deployment duy trì replica (nếu có log cũ)

9.2. Planned Worker Failover

Kịch bản đã kiểm thử. xóa Pod ứng dụng để kiểm tra điều phối sang worker2, xác nhận Pod mới Running. Sau kiểm thử, cả ba node trở lại Ready.

k8smaster — chỉ kiểm tra, KHÔNG delete lại

```
kubectl get nodes -o wide
kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
```

```
k8s@k8smaster:~$ kubectl get nodes -o wide
kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
NAME         STATUS   ROLES           AGE    VERSION   INTERNAL-IP       EXTERNAL-IP   OS-IMAGE
k8smaster    Ready    control-plane   4d5h   v1.36.4   192.168.174.146   <none>        Ubuntu 24.
worker1      Ready    <none>          4d5h   v1.36.4   192.168.174.147   <none>        Ubuntu 24.
worker2      Ready    <none>          4d5h   v1.36.4   192.168.174.148   <none>        Ubuntu 24.
NAME                              READY   STATUS    RESTARTS        AGE   IP              NODE
sample-ci-app-f44f99866-c9qt6   1/1     Running   3 (5h28m ago)   22h   10.233.125.40   worker2
```

Hình 09-02 — Pod trên worker2 và toàn bộ Nodes Ready sau failover

9.3. Hardening Container / Jenkins Credentials

Deployment chạy runAsNonRoot=true, UID/GID 1000; allowPrivilegeEscalation=false, drop ALL capabilities. Container có readiness/liveness probes và giới hạn CPU/RAM. Jenkins dùng withCredentials và secret file, không hardcode access key trong Jenkinsfile.

```
dk@dk:~/jenkins-ci-lab/sample-app$ sed -n '/securityContext:/,/capabilities:
/p' k8s/deployment.yaml
          securityContext:
            runAsNonRoot: true
            runAsUser: 1000
            runAsGroup: 1000
            allowPrivilegeEscalation: false
            capabilities:
```

```
dk@dk:~/jenkins-ci-lab/sample-app$ grep -A 35 'readinessProbe:' k8s/deployme
nt.yaml
          readinessProbe:
            httpGet:
              path: /
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3

          livenessProbe:
            httpGet:
              path: /
              port: 3000
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3

          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
              ephemeral-storage: "100Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
              ephemeral-storage: "500Mi"
```

Hình 09-03 — SecurityContext, probes và resource limits

9.4. Các lưu ý tài nguyên và sự cố

Vấn đề	Đánh giá / hướng xử lý
datastore1 Free=2.87 GB	Mức trống thấp; ghi cảnh báo, không tăng disk/snapshot tùy tiện.
Worker1 >100% Allocatable (thời điểm đo)	Node Ready / MemoryPressure=False; đánh giá với free -h và kubelet reserves.
kubectl mặc định trên dk trả trang Jenkins	Kubeconfig mặc định trên dk sai; Jenkins Pipeline dùng Credential file riêng nên deploy vẫn thành công.
Một Replica trong Deployment	Có self-healing/failover nhưng không bảo đảm zero downtime.
Cloudflare VPN	Dùng cho remote Home Lab; không bắt buộc đường CI/CD.
Shutdown/Restart Recovery	Chưa thực hiện; sẽ lên kịch bản riêng sau khi hoàn tất tài liệu.

9.5. Kiểm thử phục hồi sau thời gian shutdown dài hạn (ECR Token Expiration & Recovery)

Mục tiêu: Kiểm tra khả năng phục hồi hệ thống Hybrid CI/CD sau khi môi trường lab ngừng hoạt động trên 12 giờ, đặc biệt là tình huống AWS ECR authentication token hết hạn.

Quy trình kiểm thử:

1. Shutdown môi trường lab và duy trì trạng thái tắt trên 12 giờ.
2. Khởi động lại hạ tầng VMware, Kubernetes và Jenkins.
3. Kiểm tra trạng thái Kubernetes Cluster và ứng dụng sample-ci-app.
4. Ghi nhận lỗi 403 Forbidden và ImagePullBackOff do ECR authentication token hết hạn.
5. Làm mới ecr-pull-secret, kiểm tra CronJob ecr-token-refresh và xác nhận ứng dụng phục hồi.

Kết quả:

- Kubernetes Cluster: 3/3 Nodes Ready.
- AWS ECR token hết hạn: Đã ghi nhận.
- Kubernetes image pull: Phát sinh lỗi ImagePullBackOff.
- ECR Pull Secret: Làm mới thành công.
- CronJob refresh token: Đã điều chỉnh cấu hình và kiểm tra hoạt động.
- Ứng dụng sample-ci-app: Phục hồi thành công, Deployment đạt 1/1, NodePort 30081 hoạt động.

9.6. Tối ưu Jenkins CI/CD Pipeline – Tránh Lập Build

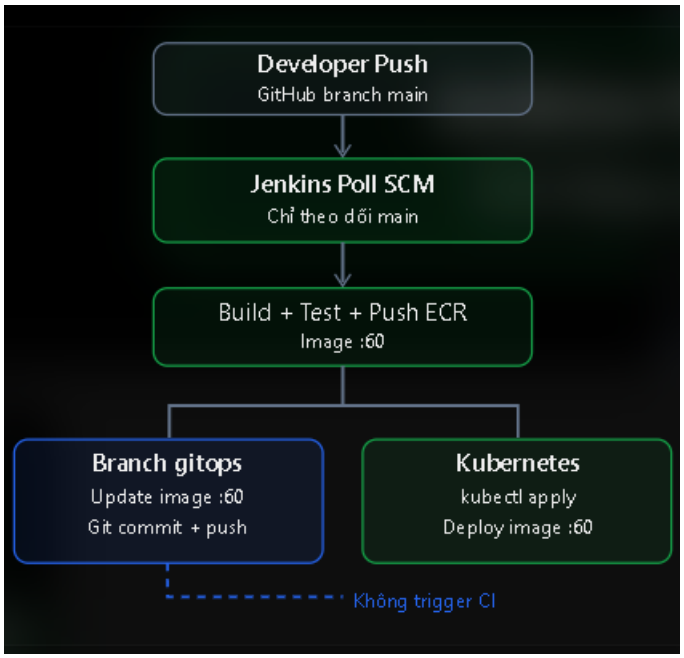
Trong mô hình ban đầu, Jenkins sử dụng chung branch main cho cả source code và GitOps Manifest.

Nguyên nhân: Jenkins sử dụng Poll SCM để theo dõi thay đổi trên main.

Khi Pipeline tự commit thay đổi image tag vào cùng branch, Jenkins có thể nhận diện đây là một thay đổi mới và tiếp tục kích hoạt Build

Hậu quả:

- Phát sinh Build không cần thiết.
- Tốn CPU, RAM và thời gian thực thi Pipeline.
- Tạo thêm Docker image trên Amazon ECR.
- Có thể gây triển khai Kubernetes liên tục ngoài ý muốn



Giải pháp: Tách riêng branch CI và GitOps.

Branch	Chức năng
main	Chứa source code và Jenkinsfile; Jenkins theo dõi để kích hoạt CI
gitops	Chứa Kubernetes Manifest được Jenkins cập nhật sau mỗi lần Build

Jenkins Poll SCM chỉ theo dõi branch main để kích hoạt CI.

Sau khi Build thành công, Jenkins cập nhật Docker image tag trong Kubernetes Manifest và commit vào branch gitops riêng biệt.

Nhờ đó, commit do Jenkins tự tạo không làm thay đổi branch main và không kích hoạt vòng lặp Build.

10. PHỤ LỤC: FILE CẤU HÌNH VÀ BẢNG LỆNH TRA CỨU

10.1. Đường dẫn file và phiên bản tham chiếu

Tài nguyên	Đường dẫn / nguồn
Repository	https://github.com/hungdoan42304/hybrid-cicd-mock-project
Revision được nghiệm thu	025fab4 (GitOps manifest image :52)
Jenkinsfile	Repository root / Jenkinsfile
Dockerfile	Repository root / Dockerfile
Deployment YAML	k8s/deployment.yaml
Service YAML	k8s/service.yaml
Jenkins host workspace	/home/dk/jenkins-ci-lab/sample-app

10.2. File k8s/deployment.yaml — bản cấu hình tại commit 025fab4

Trích từ GitHub tại revision 025fab4 — nguyên nội dung manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-ci-app
  namespace: cicd-lab

spec:
  replicas: 1

  selector:
    matchLabels:
      app: sample-ci-app

  template:
    metadata:
      labels:
        app: sample-ci-app

    spec:
      imagePullSecrets:
        - name: ecr-pull-secret
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: workload-priority
                    operator: In
                    values:
                      - primary
                      - secondary
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: workload-priority
                    operator: In
                    values:
                      - primary

      containers:
        - name: sample-ci-app
          image: 510869689558.dkr.ecr.ap-southeast-2.amazonaws.com/sample-ci-app:52
          imagePullPolicy: Always

          securityContext:
            runAsNonRoot: true
            runAsUser: 1000
            runAsGroup: 1000
            allowPrivilegeEscalation: false
            capabilities:
              drop:
                - ALL
          ports:
            - containerPort: 3000

          readinessProbe:
            httpGet:
              path: /
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3

          livenessProbe:
            httpGet:
              path: /
              port: 3000
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3

          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
              ephemeral-storage: "100Mi"
```

```
        limits:
          cpu: "200m"
          memory: "128Mi"
          ephemeral-storage: "500Mi"
```

10.3. File Dockerfile, service.yaml và package.json

Dockerfile — nguyên nội dung tại revision 025fab4

```
FROM node:20-alpine
WORKDIR /app
ARG BUILD_NUMBER=local
ENV APP_VERSION=${BUILD_NUMBER}
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
RUN chown -R node:node /app
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

k8s/service.yaml — nguyên nội dung tại revision 025fab4

```
apiVersion: v1
kind: Service
metadata:
  name: sample-ci-app
  namespace: cicd-lab
spec:
  type: NodePort
  selector:
    app: sample-ci-app
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30081
```

package.json — nội dung tham chiếu tại revision 025fab4

```
{
  "name": "sample-ci-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js",
    "test": "node -e \"console.log('Tests passed')\""
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

10.4. Jenkinsfile — các đoạn cấu hình trọng tâm

Jenkinsfile đầy đủ được quản lý trong GitHub (revision 025fab4). Bên dưới là các đoạn lệnh xác thực cơ chế ECR, GitOps và Kubernetes Deployment; sử dụng file gốc nếu cần đưa trọn vẹn vào phụ lục phiên bản mở rộng.

Jenkinsfile / Pipeline Overview

```
environment {
    IMAGE_NAME = "sample-ci-app"
    AWS_REGION = "ap-southeast-2"
    ECR_REPOSITORY = "sample-ci-app"
    GIT_BRANCH_NAME = "main"
    GIT_MANIFEST = "k8s/deployment.yaml"
}
// Các stage: Checkout → Check Commit → Install dependencies
// → Test → Build Docker image → Push Amazon ECR
// → Update GitOps Manifest → Refresh ECR Pull Secret
// → Deploy Kubernetes
```

Jenkinsfile / Deploy Kubernetes

```
withCredentials([
  file(credentialsId: 'k8s-vmware-deployer', variable: 'KUBECONFIG')
]) {
  sh '''
    kubectl apply -n cicd-lab -f "$GIT_MANIFEST"
    kubectl rollout status deployment/sample-ci-app \
      -n cicd-lab --timeout=180s
    kubectl get pods -n cicd-lab -l app=sample-ci-app -o wide
  '''
}
```

10.5. Bảng lệnh tra cứu nhanh

Mục đích	Lệnh an toàn
Tình trạng Nodes	kubectl get nodes -o wide
Workloads	kubectl get deployment,svc,pods,cronjobs -n cicd-lab -o wide
Rollout	kubectl rollout status deployment/sample-ci-app -n cicd-lab
Node resource	kubectl top nodes
Jenkins Container	docker ps --filter name=jenkins
GitOps commit	git log -1 --oneline origin/main
HTTP App	curl -I http://192.168.174.148:30081/
AWS Registry	aws ecr describe-repositories --repository-names sample-ci-app --region ap-southeast-2

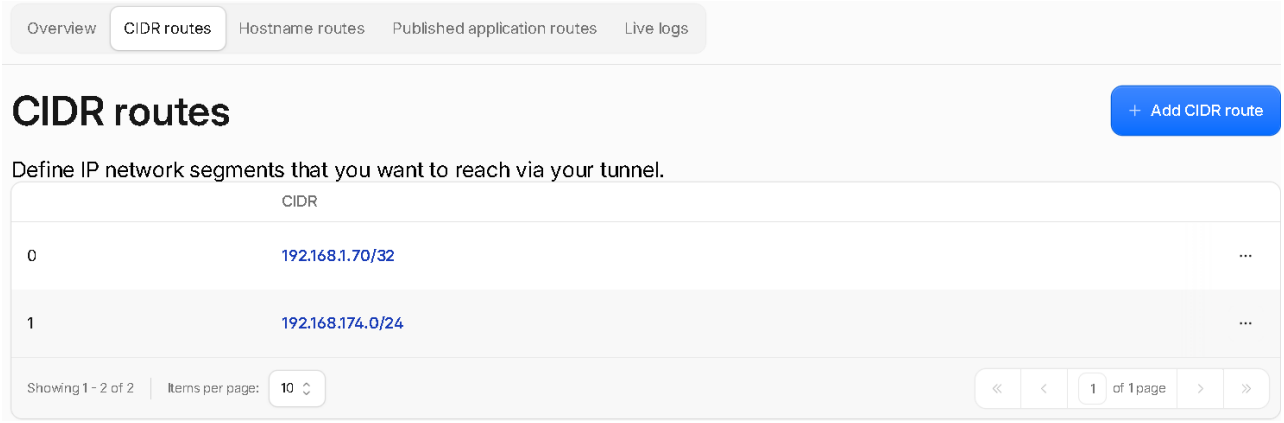
11. Kết nối mạng Hybrid thông qua Cloudflare Zero Trust/WARP

Mục tiêu: Thiết lập kết nối private từ laptop chạy Jenkins đến hệ thống VMware/Kubernetes trên PC, phục vụ quản trị và triển khai ứng dụng trong mô hình Hybrid CI/CD.

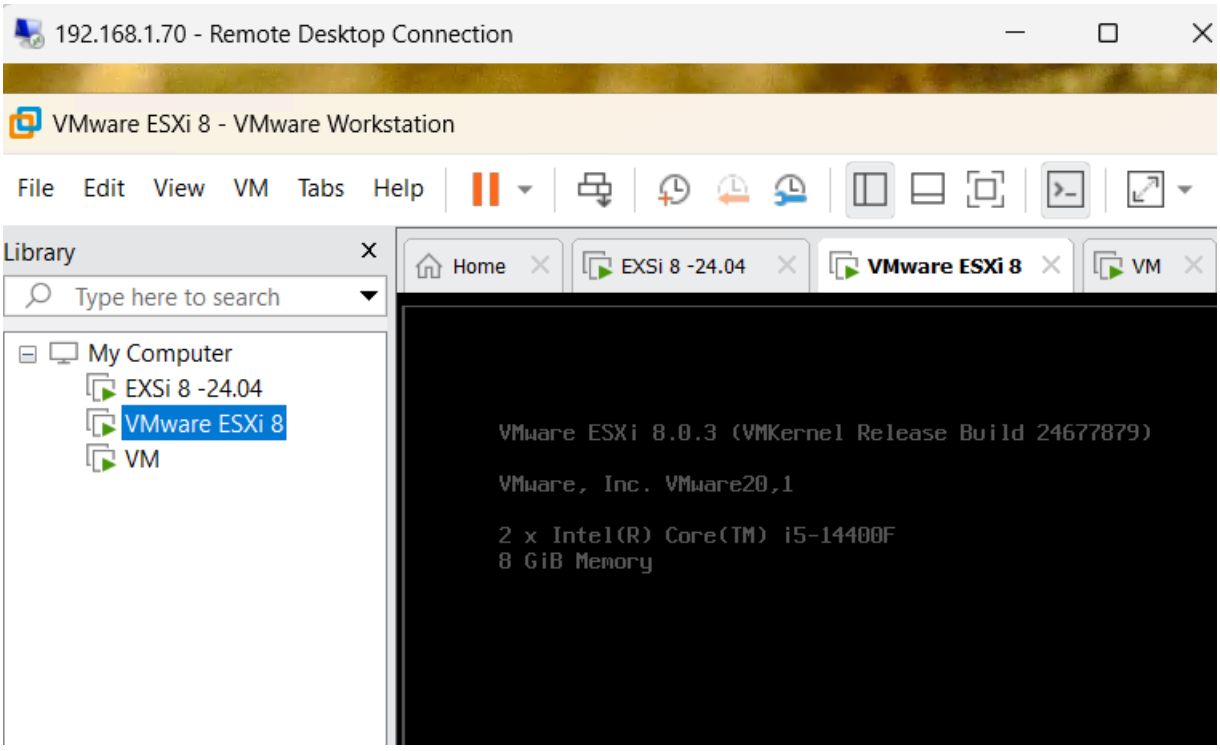
11.1 Các thành phần triển khai:

Thành Phần	Cấu hình / Vai trò
Laptop — Jenkins Server	dk — 192.168.232.170
Mạng laptop	192.168.232.0/24
Cloudflare WARP	Client trên laptop, kết nối Zero Trust
Cloudflare Tunnel	HomeLab-PC, connector cloudflared trên PC
Private Network Route	192.168.174.0/24
Kubernetes Control Plane	192.168.174.146:6443

Nguyên lý hoạt động: Laptop sử dụng Cloudflare WARP để truy cập dải mạng private 192.168.174.0/24.



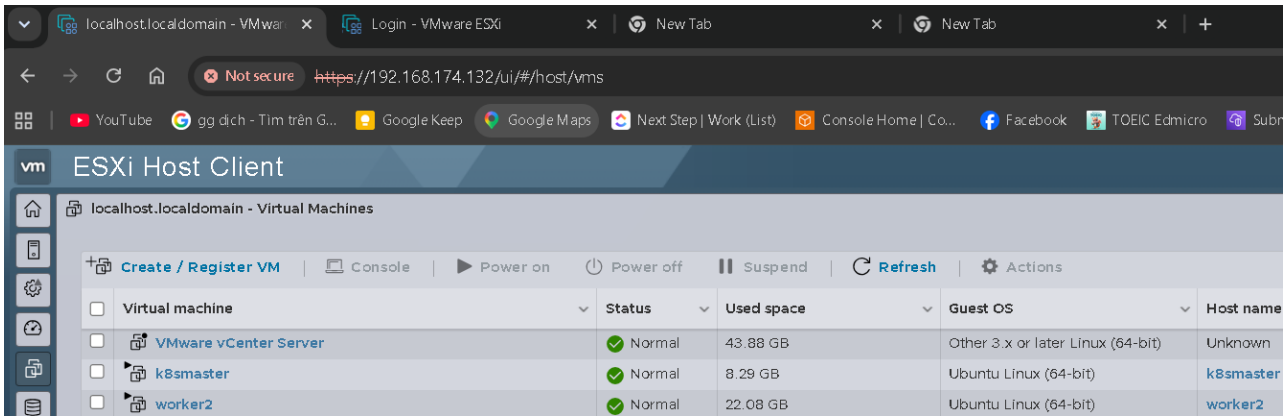
Cloudflare định tuyến lưu lượng qua tunnel HomeLab-PC đến connector cloudflared chạy trên PC.

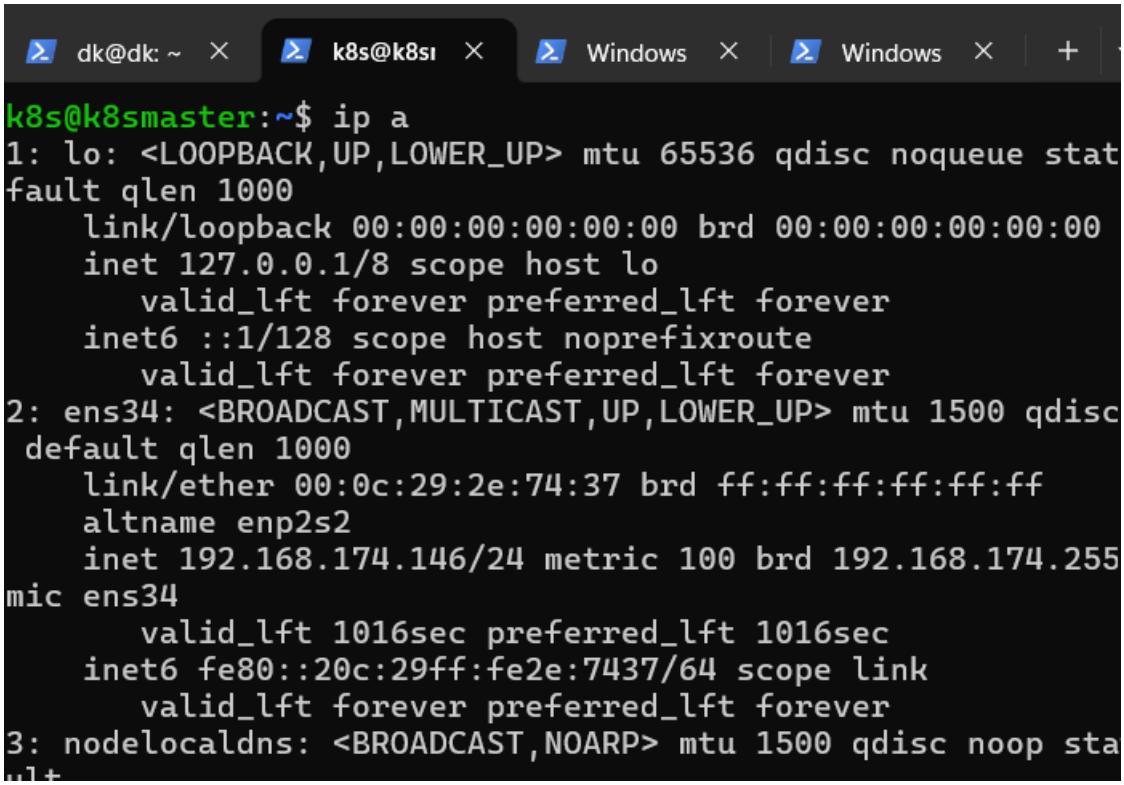


Kết quả: Hệ thống Cloudflare WARP/Tunnel đã được kiểm thử truy cập từ laptop đến môi trường VMware Lab, bao gồm cả khi laptop sử dụng mạng ngoài.

Giải pháp cho phép truy cập tài nguyên nội bộ mà không cần công khai Kubernetes API hoặc các dịch vụ ra Internet. Đây là nền tảng kết nối phục vụ quản trị từ xa và tích hợp môi trường Hybrid.

Từ đây, lưu lượng được chuyển tiếp đến các máy ảo trong VMware Lab.
Cho phép truy cập các dịch vụ của PC trực tiếp trên laptop như ssh, platform ESxi…





Bổ sung/Hướng phát triển:

VPN turnel 2 chiều từ laptop ⇔ PC,
thay thế kết nối trực tiếp bằng VPN từ Jenkin → Kubernetes API
Máy vsphere cấu hình HA, DRS cho site on-prem,

Thứ tự cải tiến nên thực hiện

Giai đoạn 1 — Làm pipeline an toàn và ổn định

Xác nhận luồng GitHub → Jenkins → ECR → Kubernetes; kiểm tra xác thực ECR từ worker, giới hạn quyền Kubernetes, bổ sung readiness/liveness probes, requests/limits và xử lý khả năng pipeline tự kích hoạt lại khi Jenkins commit manifest.

Giai đoạn 2 — Hoàn thiện CI/CD

Cấu hình GitHub webhook thay cho polling định kỳ nếu môi trường mạng cho phép. Thêm security scanning, tag image theo commit SHA, lưu log build, kiểm tra trạng thái rollout và xác nhận rollback.

Giai đoạn 3 — Chuyển sang GitOps

Tạo repository deployment riêng, dùng Pull Request để kiểm soát việc thay đổi manifest và triển khai Argo CD. Jenkins tập trung vào CI và xuất bản image; Argo CD quản lý đồng bộ CD.

Giai đoạn 4 — Hướng production

Bổ sung giám sát và cảnh báo, backup cấu hình/etcd, kiểm tra phục hồi sau sự cố, phân bố VM hợp lý giữa các ESXi host, thiết kế control plane HA nếu SLA yêu cầu, cùng quy trình quản lý thay đổi và phê duyệt.